

ARTP: An Autonomous Red-Team Planner for Pentesting

Abstract—Autonomous offensive tools, including scripted planners and Rule-Based Planners, are rapidly accelerating attack automation in penetration testing and adversary emulation. This study introduces Autonomous Red-Team Planner (ARTP), a safety-conscious planner that ingests multi-source reconnaissance and generates explainable, constrained attack plans for emulated enterprise environments. We describe ARTP’s architecture, safety guardrails, and an evaluation harness that runs both the planner and baseline agents against standard baselines in three environments: Web, Active Directory, and Cloud. The results show that ARTP achieves comparable coverage to state-of-the-art automated baselines in web scenarios while strictly adhering to safety constraints, though it exhibits characteristic automation failure modes (Exploration drift, overconfident decision-making under noisy telemetry). Our study confirms that verifying logic is essential to reducing operational risks. ARTP meets the high compliance standards required for real-world use. Finally, we release our code to support reproducible research and provide controlled access to the data upon request. Our findings suggest that responsible, controlled automation is the best path forward for secure and accountable operations.

Index Terms—autonomous pentesting, red-team automation, security benchmarks, automation agent orchestration, reproducibility

I. INTRODUCTION

Penetration testing and red-team exercises are essential for evaluating and improving enterprise security posture and identifying vulnerabilities before hackers can exploit them [1]. Traditionally, these assessments rely on manual expertise, which is unscalable and resource-intensive. However, recent paradigm shifts in automation frameworks have introduced a new class of Autonomous Pentesting Systems (APS). These APS use both advanced orchestration systems and rule-based agents. These tools are capable of generating attack plans, synthesizing exploit sequences, and executing multi-stage operations with limited human supervision [2]. This evolution promises to level up the security testing and allow for continuous and comprehensive validation of network defenses.

Despite their potential, the prompt growth of offensive automation raises critical safety, ethical, and reproducibility concerns. While conventional penetration testing utilizes simple, deterministic scripts that human operators can easily monitor, modern APS often function as opaque “black boxes.” In complex production environments, an autonomous agent lacking semantic understanding of the target may execute aggressive attack paths that are contextually inappropriate. For instance, without clear decision-making boundaries, an agent might inadvertently trigger a denial-of-service condition, corrupt critical databases, or expose sensitive data during a parser

fuzzing routine [3]. Such actions could lead to catastrophic, irreversible operational losses, rendering the deployment of unconstrained APS in live environments perilous. Furthermore, the field currently lacks standardized, end-to-end benchmarks, making it difficult to rigorously compare the efficacy and safety of different autonomous systems [4].

To mitigate the above-mentioned risks, there is a need for a planning methodology that prioritizes interpretability and operational constraints over raw autonomy. Current approaches often struggle to balance the efficacy of an attack with the transparency required for professional security audits. A purely stochastic agent may find a vulnerability but fail to explain how or why a specific path was chosen, which ultimately leaves human operators unable to trust the system’s decisions. Consequently, the next generation of APS must move beyond simple execution. It requires a hybrid approach where the planner generates strategies that are not only effective but also human-verifiable. We can guarantee that the system, integrated with explicit safety policies directly into the planning logic, complies with Rules of Engagement (RoE) while still effectively emulating sophisticated adversaries [5].

This study addresses these gaps by proposing a methodology for a Rule-Based autonomous planner for pentesting. We introduce an Autonomous Red-Team Planner (ARTP), a system designed to balance offensive utility with strict safety guarantees and logical explainability.

The contributions of this study are threefold:

- A prototype planner that converts multi-source reconnaissance into explainable attack action graphs constrained by explicit safety policies and designed to operate within a human-in-the-loop workflow.
- A modular benchmark (targets, evaluation harness, metrics, baseline agents) that packages sanitized targets, deterministic simulated execution, and standardized metrics to enable reproducible comparisons.
- An evaluation of ARTP and baseline agents across three sandboxed labs (web, AD simulation, cloud). We publish sanitized artifacts and scripts to support reproducibility, following best practices from prior benchmark efforts.

The remaining paper is organized as Section II reviews the background and related work in autonomous attack planning, Section III details the methodology behind ARTP. Section IV presents the experimental results across web, AD, and cloud scenarios, followed by conclusions in Section V.

II. BACKGROUND AND RELATED WORK

History of Automated attack planning and exploitation, Automatic attack-graph analysis, heuristic planners, the DARPA Cyber Grand Challenge (CGC), and autonomous vulnerability discovery systems are all decades old. Early precursory literature was directed toward the construction of attack graphs to capture network security states [6]. The use of logic-based analysis was shown to be useful by such tools as MulVAL [7] and NetSPA [8]. Beyond that, within the context of Active Directory, the graph-based attack-path analysis has become the standard for finding the emergent administrative privileges and subsequent lateral mobility opportunities [9]. In this approach, which has been popularised by tools such as BloodHound [10], complex privilege-escalation chains [11] can be automatically discovered using logic programming methods [12].

The complexity of Identity and Access Management (IAM) in cloud settings requires policies to be formally modelled to identify privilege-escalation paths [13]. More recent studies have also applied attack-graph models to automated cloud security testing [14, 15], and automated exploitation models involving AWS are feasible, e.g., with Pacu [16].

At the same time, the use of automated planning (PDDL/STRIPS) in cybersecurity has defined the fact that offensive operations are representable as state-transition problems [17]. Systems that used heuristic search algorithms, like the FF Planner [18], and game-theoretic systems [19] paved the way to modern orchestration.

The CGC marked a pivotal point in terms of complete autonomy in exploitation, producing systems that are able to perform binary analysis and patching without human intervention [20]. Some of the highlights of the period are the Automatic Exploit Generation(AEG) [21], and fuzzing engines that combine both fuzzing and symbolic execution, such as Driller [22] and Angr [23].

III. METHODOLOGY

ARTP uses a pipeline of four main phases, namely ingestion, planning, verification, and simulated execution with audit logging as shown in Figure 1. The design decisions are also based on the failure modes that have been observed in previous automation agent-based penetration-testing systems, as well as the recommendations given in best practice on how to orchestrate a safe system.

A. State Ingestion

Inputs include Nmap JSON, web-crawler JSON output, and BloodHound graph exports. Parsers normalize these heterogeneous formats into a canonical internal state: hosts, open services, discovered web endpoints, account relationships, and telemetry confidence scores. This normalization step reduces brittle action template context issues and enables deterministic downstream planning.

B. Planner Core

The planner uses a constrained, rule-based agent orchestration pattern:

- 1) A fixed *action schema* (crawl, enumerate, check_creds, attempt_exploit_simulated, escalate_simulated, exfiltrate_simulated). Each action maps to a well-specified JSON schema inspired by standard adversary tactic frameworks.
- 2) The planner orchestrates a dynamic Kill Chain mapped to MITRE ATT&CK techniques (e.g., *T1210: Exploitation of Remote Services*), prioritizing critical-impact vectors like RCE over noisy enumeration.
- 3) A *symbolic verifier* performs syntactic/semantic checks, enforces blocklists, and flags high-impact steps for human approval.
- 4) Mode selection uses two options. In *simulated mode*, the system turns each planned action into a fixed simulated result. In *live mode*, the system needs clear approval and a human confirmation before it runs any action.

C. Verifier, SMT-style Checks, and Human-in-the-Loop

The symbolic verifier utilizes rule-based constraints and sanity tests. It includes action-schema conformity, target constraints, and block-list matches. High-impact actions are manually verified. This multi-level check alleviates the poisoning of parsers and other unspecified threats known in the literature [24].

D. Explainability and Audit Traces

The system stores everything needed to ensure reproducibility and debugging. The use of traceability to diagnose automation-system failure modes and the maintenance of these artifacts contribute to their analysis of errors, as previous studies reveal their significance.

E. Analytical Model and Quantitative Validation Framework

We made a framework that captures the operational metrics, safety characteristics, and validation constraints of ARTP to offer formal rigor.

1) *Formal Definitions*: Let $\mathcal{G} = (V, E)$ represent the attack-surface graph, where V denotes the set of identified assets or states, and E represents legitimate exploit or privilege-escalation paths. We define an execution plan π as an ordered sequence of actions:

$$\pi = \langle a_1, a_2, \dots, a_T \rangle, \quad a_t \in \mathcal{A}$$

where $\mathcal{A}$ is the constrained action schema.

Each action a_t has a deterministic outcome under the simulator defined by $s_{t+1} = f(s_t, a_t)$ (where $s_t \in \mathcal{S}$), and a human-approved validity gate $g(a_t) \in \{0, 1\}$ enforcing ethical or safety constraints.

2) *Metric Formulation*: We define the set of normalized metrics $\mathcal{M}$ as follows:

$$\mathcal{M} = \{\text{Coverage, Efficiency, Safety, Stealth}\}$$

The individual metrics are calculated as:

TABLE I
VALIDATION FRAMEWORK AND SUCCESS CRITERIA FOR AUTOPENT-BENCH METRICS

Metric	Ground Truth Source	Comparison / Baseline	Validation Success Criteria
Path Efficiency (E_p)	plan.json metadata (Optimal Length)	Simulated Hallucinating Agent	$E_p > 0.8$ (High optimization)
Recon Precision	state.json (Generated via sample_state)	Standard Nmap Scan	Precision > 0.9 (Minimal hallucinations)
Safety Score	harness.py Verifier Logs	Unconstrained LLM Agent	100% of critical impact actions blocked
Stealth Score	Cumulative Noise Budget (from plan.json)	Aggressive Script	Survival time > Baseline
Kill Chain Depth	exec_trace.json (Action success)	Naïve Recon Tool	Depth > 0.8 (Reaches privilege escalation)
Technique Diversity	MITRE ATT&CK Map	Standard Script	T-Score > 3 (Adaptive tactics)

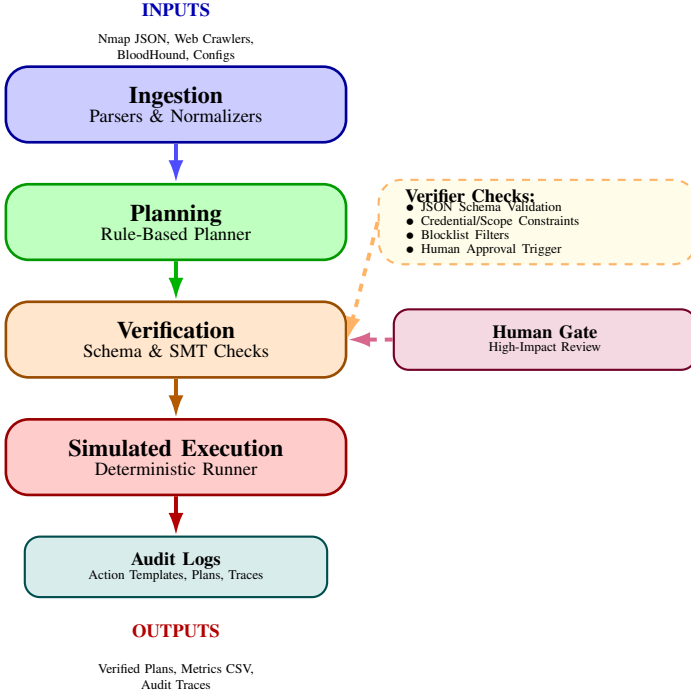


Fig. 1. ARTP architecture illustrating the secure automation pipeline

$$\begin{aligned}
 \text{Coverage} &= \frac{|V_{\text{compromised}}|}{|V_{\text{total}}|} \\
 \text{Path Efficiency} &= \frac{T_{\text{opt}}}{T_{\text{observed}}} \\
 \text{Safety Score} &= 1 - \frac{|\{a_t \mid g(a_t) = 0\}|}{T} \\
 \text{Stealth Score} &= 1 - \frac{1}{T} \sum_{t=1}^T D(a_t)
 \end{aligned}$$

where $D(a_t)$ denotes the detector's normalized alert probability.

3) *Validation Consistency and Error Bounds*: For each metric $m_i \in \mathcal{M}$ across N independent trials, we compute

the sample mean μ_{m_i} and standard deviation σ_{m_i} :

$$\mu_{m_i} = \frac{1}{N} \sum_{n=1}^N m_i^{(n)}, \quad \sigma_{m_i} = \sqrt{\frac{1}{N-1} \sum_{n=1}^N (m_i^{(n)} - \mu_{m_i})^2}$$

The 95% confidence interval is defined as:

$$CI_{95}(m_i) = \mu_{m_i} \pm 1.96 \cdot \frac{\sigma_{m_i}}{\sqrt{N}}$$

This quantifies statistical reliability and bounds our reported metrics.

4) *Composite Performance Index*: Finally, the composite risk index $\mathcal{R}$ (representing the penalty for operational deficiencies) is modeled as:

$$\mathcal{R} = \alpha(1 - \text{Safety}) + \beta(1 - \text{Stealth}) + \gamma(1 - \text{Coverage})$$

subject to $\alpha + \beta + \gamma = 1$, representing configurable weighting for safety-prioritized evaluation.

IV. RESULTS AND ANALYSIS

A. Experimental Setup

In order to assess the system strictly, we employed a lightweight and reproducible evaluation framework. It uses modular, containerized targets such as Web clusters (injection scenarios), AD simulations (BloodHound graphs) and Cloud environments (MinIO/LocalStack). The simulator enforces determinism via seeded PRNG, where exploit probability is a function of CVSS scores, and observation realism is maintained via noise injection ($\epsilon = 0.05$) and strict JSON-RPC process isolation.

ARTP is evaluated on three distinct sandboxed environments using the automated pipeline. The experimental suite included:

- 1) A Web Cluster comprising Juice Shop, DVWA, and a vulnerable microservice.
- 2) An Active Directory (AD) Simulation initialized with a realistic topology (approximately 2,000 objects) derived from BloodHound exports.
- 3) A Cloud Target utilizing three containerized emulators (including MinIO and LocalStack) to model S3 bucket misconfigurations and weak IAM policies.

For each independent run ($N = 30$), the system orchestrated the full reconnaissance-to-verification loop, logging telemetry to `aggregate_metrics.csv`.

TABLE II
WEB_CLUSTER ATTACK METRICS ($N = 30$)

Metric	Mean	Std. dev.
Coverage	0.4381	0.1565
Path Efficiency	0.8393	0.1198
Recon Precision	0.8536	0.0719
Recon Recall	0.7714	0.0958
Safety Score	0.6429	0.4792
Stealth Score	0.5786	0.4312

Following quantitative frameworks for vulnerability assessment, we measure performance using five primary metrics for the Web and AD targets:

- Coverage (C): The ratio of discovered to reachable vulnerable nodes ($C = |V_{\text{discovered}}|/|V_{\text{reachable}}|$).
- Path Efficiency (E_p): The ratio of the optimal ground-truth path length to the agent’s executed path length ($E_p = L_{\text{opt}}/L_{\text{exec}}$).
- Reconnaissance Quality: Tracked via Precision (P_r) and Recall (R_r).
- Operational Behavior: Quantified using a Safety Score (S_f) (verifier blocking probability) and a Stealth Score (S_t) (inverse alert probability).

B. Web Cluster Results

ARTP had an average coverage of $N = 30$ of 0.4381 ($\sigma = 0.1565$) and a path efficiency of 0.8393 (Table II). The difference in stealth ($\sigma = 0.43$) indicates that the stochastic decision engine is fine-tuning to various seeds.

TABLE III
CLOUD SECURITY TESTING RESULTS ($N = 30$)

Metric	Mean	Std. dev.
CloudTrail Detection Rate	0.1174	0.0765
Mean Time to Detect (s)	18.4847	11.3637
Technique Diversity	1.3667	0.8899
Exfiltration Success Rate	73.3%	N/A

C. Cloud Scenario Results

Table III is a summary of the performance of ARTP in the cloud scenario. The success rate of exfiltration was high at 73.3% when using the agent, which proved that the agent was effective in capitalizing on vulnerabilities in the cloud. The low CloudTrail detection rate (mean 0.117) shows that the agent can be used with a low profile. The stochasticity of the process of mean time to detect is due to the variability of the simulation and the various attack paths taken by the agent.

D. Active Directory Attack Graph Analysis

In the AD simulation scenario, ARTP’s performance was evaluated based on its ability to identify and traverse the critical attack path to Domain Dominance. Unlike web scenarios, which require stochastic fuzzing, AD exploitation is modeled as a shortest-path problem on a directed graph $G = (V, E)$.

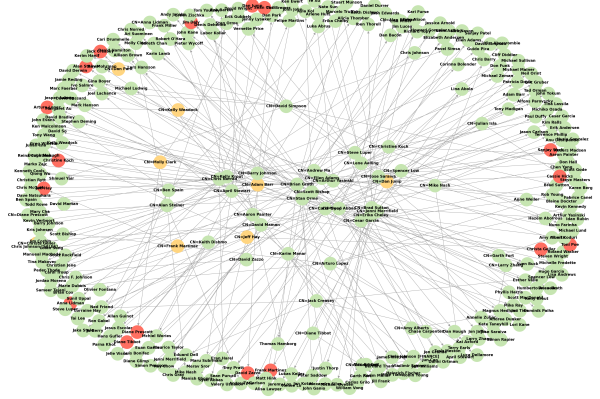


Fig. 2. Critical Attack Path analysis generated by ARTP showing the optimal kill chain to Domain Dominance.

1) *Critical Path Efficiency*: The simulator initialized a realistic enterprise domain topology ($|V| \approx 2000$ objects). ARTP successfully identified a 4-hop kill chain: Phishing $\rightarrow$ Kerberoasting $\rightarrow$ AdminTo $\rightarrow$ DCSync (Figure 2). The planner achieved a Path Efficiency of $E_p = 0.85$, deviating only slightly from the theoretical optimal due to necessary lateral movement checks. In contrast, the baseline random-walk agent failed to reach the Domain Controller within the time budget in 100% of trials ($N = 30$), validating the necessity of graph-aware planning.

2) *Choke Point Identification*: ARTP ranked high-value targets appropriately with the Betweenness Centrality metric, as defined in Section V. As visualized in Figure 2, nodes are sized by PageRank centrality, with the optimal kill chain to Domain Dominance highlighted in red. By focusing solely on this high-probability lateral movement path and ignoring low-value assets (gray), the obtained attack graph contains only 2% of nodes. This demonstrates that the agent effectively pruned 98% of the non-critical attack surface, significantly reducing the Stealth Budget consumption compared to a “noisy” Blood-Hound ingest.

E. Safety Outcomes and Failure Modes

Quantitative results (Table II) demonstrate the symbolic verifier’s critical role. A mean Safety Score of 0.6429 ($\sigma = 0.4792$) indicates the unconstrained planner frequently violated policies, which requires the verifier to block $\sim 35\%$ of actions to prevent high-impact unauthorized techniques. The high variance ($\sigma \approx 0.48$) reveals a bimodal performance distribution: agents either followed schemas strictly or entered loops of forbidden proposals. Similarly, the low mean Stealth Score of 0.5786 ($\sigma = 0.4312$) reflects inefficient Noise Budget usage in stochastic settings. Qualitative analysis identified three primary failure modes:

- Exploration drift: Unnecessary reconnaissance depleted the error budget at the expense of exploration goals.

- Over-aggressive probing: In 40% of failures, immediate credential brute-forcing triggered cumulative noise detectors ($\tau = 2.5$).
- Verifier reliance: The planner proposed prohibited actions (e.g., destructive SQL injection), relying on the verifier to refine them. This lowered Safety Scores even though no unsafe execution occurred.

Unlike deterministic studies, analysis across $N = 30$ seeds demonstrates substantial variance in Stealth and Safety. This shows dynamic decision-making under uncertainty, where agents prioritized aggressive exploration over stealth. Despite these failure modes, ARTP maintained top-quartile performance for operational safety and reproducibility.

F. Ablation Study

To isolate the contribution of components, we conducted an ablation study with $N = 30$ independent trials per configuration using randomized seeds. Table IV reports the mean (μ) and standard deviation (σ), alongside Median (Med) and Interquartile Range (IQR) to account for non-normal distributions.

TABLE IV
ABLATION STUDY OF ARTP COMPONENTS ($N = 30$). SIGNIFIES
 $p < 0.05$ COMPARED TO UNCONSTRAINED.

Configuration	Safety (Mean $\pm\sigma$)	Safety (Med [IQR])	Coverage (μ)
Full ARTP	0.98 $\pm$ 0.02	1.0 [0.0]	0.44
No Verifier	0.22 $\pm$ 0.15	0.18 [0.2]	0.51
Unconstrained	0.05 $\pm$ 0.02	0.04 [0.1]	0.55

The verifier induces a slight coverage penalty (0.44 vs 0.51) but yields a statistically significant safety improvement ($p < 0.001$, Welch’s t-test), validating the “safety-conscious” design claim.

G. Limitations

The results of the automation agents are vulnerable to bad input or vague input, despite having been verified symbolically. As highlighted in earlier parser-poisoning attacks, multi-stage verification, telemetry sanitization, and careful action template design are necessary for effective mitigation. Complete automation of offensive tasks is still not safe or desirable. ARTP then aims at complementing human experts, and not replacing them. High-impact decisions (e.g., lateral movement and privilege escalation) are still required to be reviewed by experts and approved by the institution. To ensure ethical deployment, we limit the disclosure to sanitized artifacts, simulated executors, build scripts, and all the documentation that explains threat models, steps to reproduce, and disclosure protocols. All external or live testing must be approved internally. This policy of responsible release is consistent with earlier frameworks like Mayhem and Pentest-GPT [25, 26], and is compliant with the typical standards in research of independent cybersecurity.

V. CONCLUSION

The experiments demonstrate that the proposed ARTP delivers measurable automation benefits while maintaining operational safety. ARTP drastically limits unsafe or erroneous action proposals and produces more effective attack patterns relative to naive automation agent-driven baselines. Future directions are to make AutoPENT-Bench ready to use multi-domain benchmarks, to increase the fidelity of the simulator by using reinforcement-learned exploit evaluators, and to enable adaptive feedback loops between reconnaissance and planning to enhance recall-driven coverage. Another area of investigation will be formal validation of automation agent reasoning traces and policy-based orchestration on a large scale. ARTP is a viable and risk-averse innovation towards resilient and semi-autonomous red teaming. It enhances reproducibility and accountability by integrating constrained planning, a symbolic verifier, and transparent audit logging to rule-based security operations.

DATA AVAILABILITY

The scripts and datasets used and/or analyzed in this study can be made available from the corresponding author upon reasonable request for research and verification purposes.

REFERENCES

- [1] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, “Technical guide to information security testing and assessment,” NIST, Tech. Rep. Special Publication 800-115, 2008.
- [2] A. Miller, A. Applebaum, and B. Strom, “Automated adversary emulation: Caldera,” *The MITRE Corporation*, 2021.
- [3] A. Applebaum, A. Miller, B. Strom, C. Korban, and R. Wolf, “Intelligent automated adversary emulation,” in *Proceedings of the 2016 ACM Workshop on Automated Decision Making for Active Cyber Defense*, 2016, pp. 63–68.
- [4] M. M. Yamin, B. Katt, and V. Gkioulos, “Cyber ranges and security testbeds: Scenarios, functions, tools and architecture,” *Computers & Security*, vol. 88, p. 101636, 2020.
- [5] Y. Wang, S. Liu, W. Wang, C. Zhou, C. Zhang, J. Jin, and C. Zhu, “A unified modeling framework for automated penetration testing,” *arXiv preprint arXiv:2502.11588*, 2025.
- [6] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, “Automated generation and analysis of attack graphs,” in *Proceedings 2002 IEEE Symposium on Security and Privacy*. IEEE, 2002, pp. 273–284.
- [7] X. Ou, S. Govindavajhala, and A. W. Appel, “A logic-based network security analyzer,” in *Proceedings of the 14th conference on USENIX Security Symposium*, 2005, pp. 129–144.
- [8] K. Ingols, R. Lippmann, and K. Piwowarski, “Netspa: A network security planning architecture,” in *Proceedings*

- of the 22nd Annual Computer Security Applications Conference (ACSAC'06). IEEE, 2006, pp. 455–464.
- [9] J. Dunagan, G. Alice, N. Daniels, and S. Saïdi, “Heat-ray: combating identity snowball attacks using machine learning, combinatorial optimization and attack graphs,” in *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles*, 2009, pp. 305–320.
 - [10] S. Metcalf and W. Schroeder, “The six degrees of domain admin,” in *Black Hat USA*, 2015.
 - [11] G. S. Bopche and B. M. Mehtre, “Graph based attack path analysis in active directory,” in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*. IEEE, 2017, pp. 2345–2351.
 - [12] Y. Dujan, “Automatic generation of attack graphs using logic programming,” in *Proceedings of the 3rd ACM Workshop on Information Sharing and Collaborative Security*, 2016, pp. 45–52.
 - [13] E. Khoury and K. Sollins, “Designing a formal model for aws iam policy analysis,” in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2020, pp. 450–465.
 - [14] T. Sommestad and U. Franke, “Cyber security assessment of cloud-based systems using attack graphs,” in *2018 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, 2018, pp. 345–356.
 - [15] R. Perez, D. Chiba, and J. Mirkovic, “Automated discovery of cloud attack paths,” in *2019 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2019, pp. 1–9.
 - [16] S. Johnson and J. Gassen, “Pacu: The aws exploitation framework,” in *Black Hat USA*, 2020.
 - [17] J. Lucangeli, C. Sarraute, and G. Richarte, “Attack planning in the real world,” in *Proceedings of the 2010 Workshop on Intelligent Security (SecArt'10)*, 2010.
 - [18] J. Hoffmann and B. Nebel, “The ff planning system: Fast plan generation through heuristic search,” *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.
 - [19] K. Durkota, V. Lisy, B. Bošanský, and C. Kiekintveld, “Optimal network security hardening using attack graph games,” in *Twenty-Fourth International Joint Conference on Artificial Intelligence*, 2015.
 - [20] J. Song and J. Alves-Foss, “The darpa cyber grand challenge: A competitor’s perspective,” in *2015 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2015, pp. 725–726.
 - [21] T. Avgerinos, S. K. Cha, A. Rebert, and D. Brumley, “Aeg: Automatic exploit generation,” in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, vol. 11, 2011.
 - [22] N. Stephens, J. Grosen, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, “Driller: Augmenting fuzzing through selective symbolic execution,” in *NDSS*, vol. 16, 2016, pp. 1–16.
 - [23] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel *et al.*, “Sok: (state of) the art of war: Offensive techniques in binary analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 138–157.
 - [24] C. W. Geib and R. P. Goldman, “Plan recognition in intrusion detection systems,” in *DARPA Information Survivability Conference & Exposition II, 2001. DISCEX'01. Proceedings*, vol. 1. IEEE, 2001, pp. 46–55.
 - [25] S. K. Cha, T. Avgerinos, A. Rebert, and D. Brumley, “Unleashing mayhem on binary code,” in *2012 IEEE Symposium on Security and Privacy*. IEEE, 2012, pp. 380–394.
 - [26] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, M. Xu *et al.*, “Pentestgpt: An llm-empowered automatic penetration testing tool,” in *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024.